

20250616初步报告1

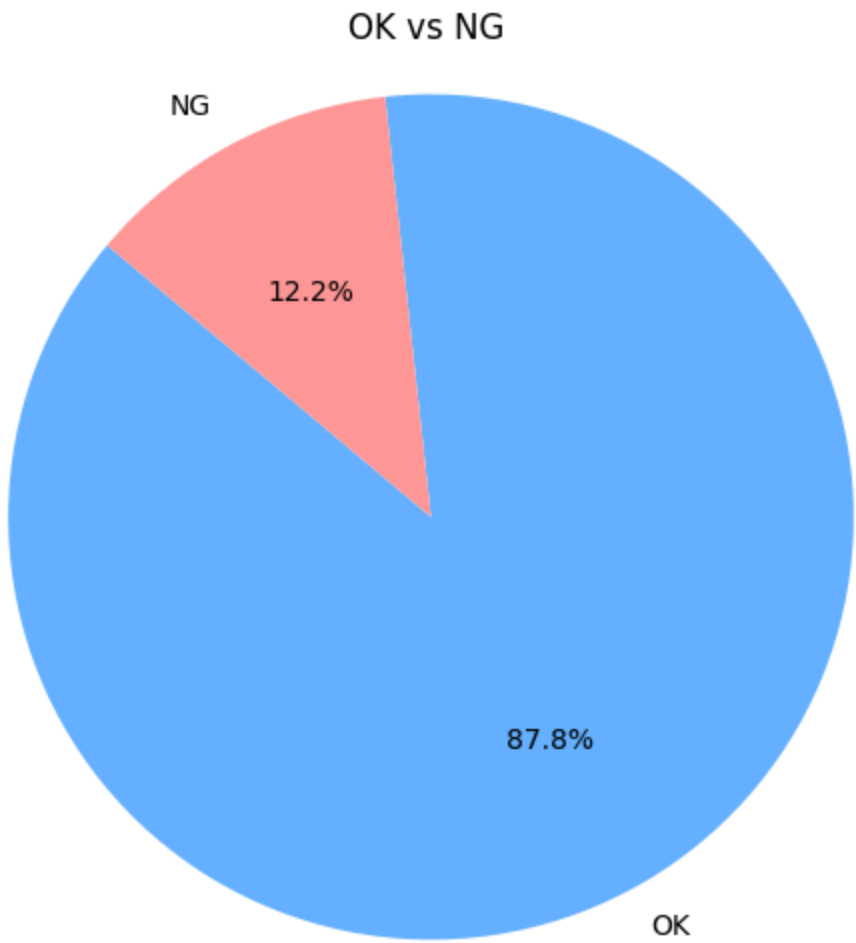
概述

本模型基于ResNet18进行训练，旨在完成工业X光图的缺陷检测。

本模型根据给定的图像输入，对图像进行二分类，分为有缺陷（OK）与无缺陷（NG）。

数据分析

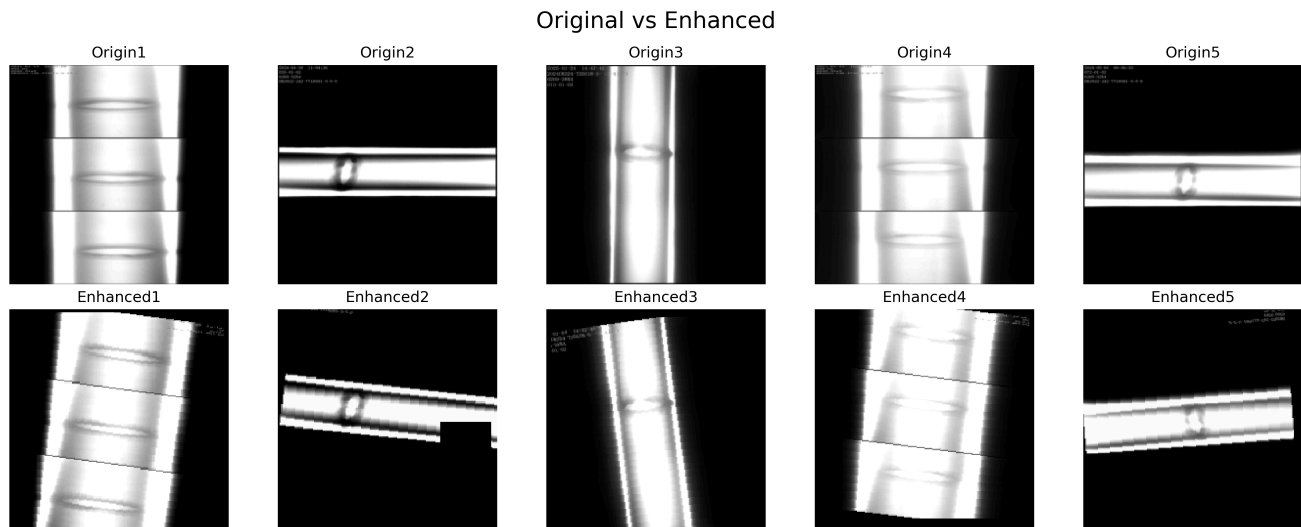
以下为给定数据集的类别分布：



数据质量问题：

数据集严重不平衡，缺陷组仅有12.2%。在后续的训练中考虑调整权重时模型着重训练缺陷组。

预处理与增强：



其中处理手段包括：

- 翻转图像
- 随机旋转图像
- 随机改变亮度，对比度，饱和度
- 轻微仿射变换
- 以50%的概率抹去部分区域

在本训练流程中不对测试集进行预处理。

模型选择与训练：

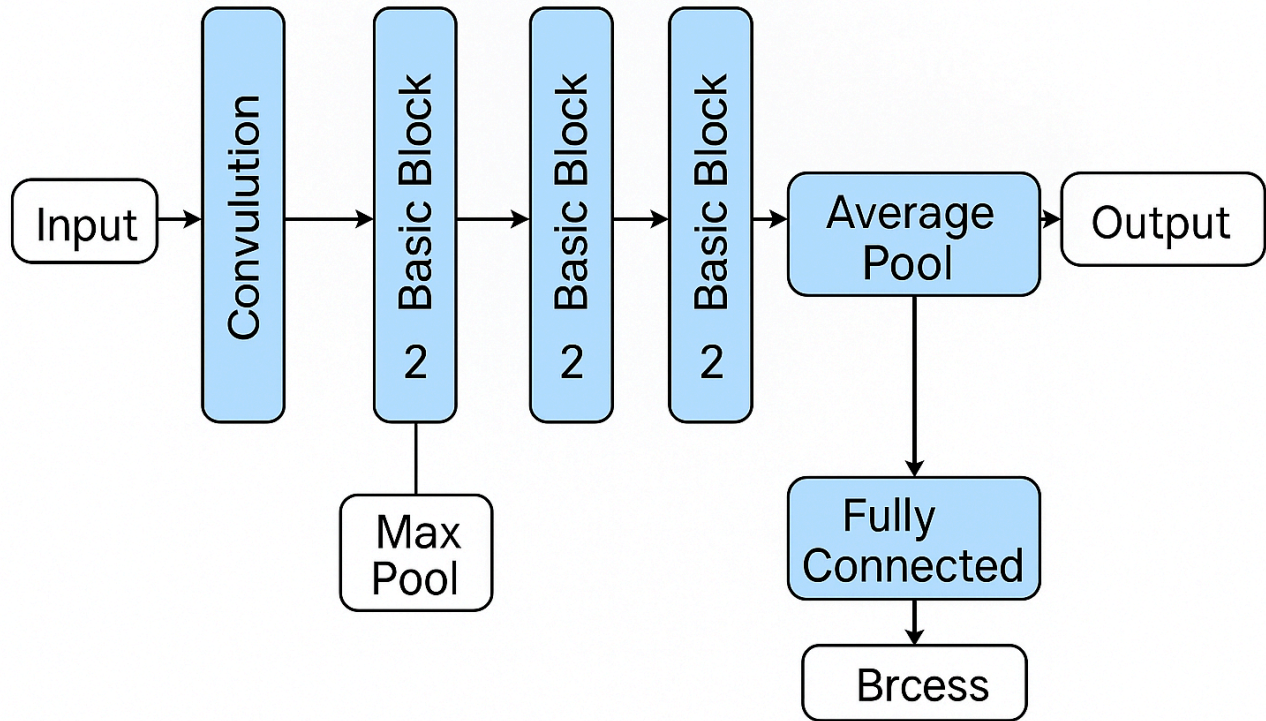
模型选择：

在工业检测中，模型承担的主要责任应该在于精确地发现缺损。因此模型的Recall指标十分重要。

数据集规模较小，且NG与OK之间严重不平衡。为了避免模型在OK集上的过拟合，模型结构应该简单，ResNet18更为轻量，能够在一定程度提高NG上的召回率（Recall），且相较于

ResNet30等中等规模的模型，有更快的收敛速度，训练成本更低。

ResNet-18



在训练神经网络时，会计算损失loss以来表示预测和实际的误差，然后用反向传播算法

$$\theta = \theta - n \nabla L(\theta)$$

其中 θ 是模型参数， n 是学习率， $\nabla L(\theta)$ 是在参数在损失函数 $L(\theta)$ 上的梯度，减号意味着我们要朝着梯度相反的方向前进。

我们需要计算每一层loss在数学意义上的梯度，进而告诉我们：要让这个误差达到最小需要让各个参数朝着哪个方向前进。

一般地，假设我们有三层神经网络，将神经网络看作n层复合函数：

训练时需要对所有参数求梯度，这是对损失函数 L 分别求参数的偏导，而根据导数的链式法则，某一层的偏导是内部所有层导数的乘积。损失函数的导数 小于1。

根据上述不难发现，梯度随着模型沿着网络的前进会越来越小，可以预见到在更深的网络中，梯度会在某一刻逼近于0，这会导致模型的参数更新停滞，模型无法再学到新的东西，这称为梯度消失。

ResNet18引入残差连接。残差连接让神经网络学习输入与输出之间的差值，使梯度更新绕开中间层，直接传递到输入端，进而解决了梯度消失的问题。

ResNet18最后的全连接层+softmax，可以修改输出通道为1或2，对二分类问题具有天然支持。

同时，ResNet18的前几层卷积操作具有更小的感知视野，能够提取更低层次的特征。

综上，ResNet18被认为是更适合该任务的模型。

训练：

考虑到数据集严重不平衡，采用FocalLoss损失函数。

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t)$$

其中， p_t 是预测为真实类别的概率， α_t 为类别权重， γ 为调制因子（控制关注难类别的程度，通常为0.2）

同时动态计算权重因子：

```
from sklearn.utils.class_weight import compute_class_weight
targets = [label for _, label in train_dataset]
weights = compute_class_weight('balanced', classes=[0, 1], y=targets)
class_weights = torch.tensor(weights, dtype=torch.float32).to(device)
```

它会根据预测标签和真实标签，更新分配给不同训练标签的权重。

引入早停，避免模型在训练集上的过拟合：

```
class EarlyStopping:

    def __init__(self, patience=20, delta=0.0001,
save_path='final_model.pth'):

        self.patience = patience

        self.counter = 0

        self.best_loss = None

        self.early_stop = False

        self.delta = delta

        self.save_path = save_path

    def __call__(self, val_loss, model):

        if self.best_loss is None:
```

```
        self.best_loss = val_loss

        self.save_checkpoint(model)

    elif val_loss > self.best_loss - self.delta:

        self.counter += 1

        if self.counter >= self.patience:

            self.early_stop = True

    else:

        self.best_loss = val_loss

        self.save_checkpoint(model)

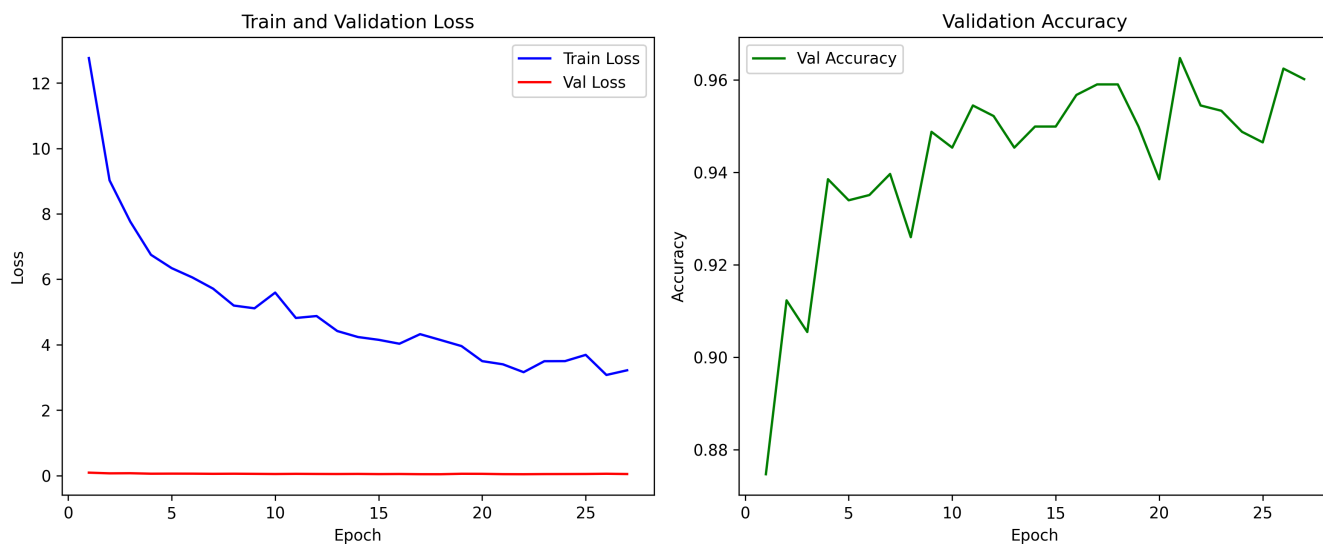
        self.counter = 0

def save_checkpoint(self, model):

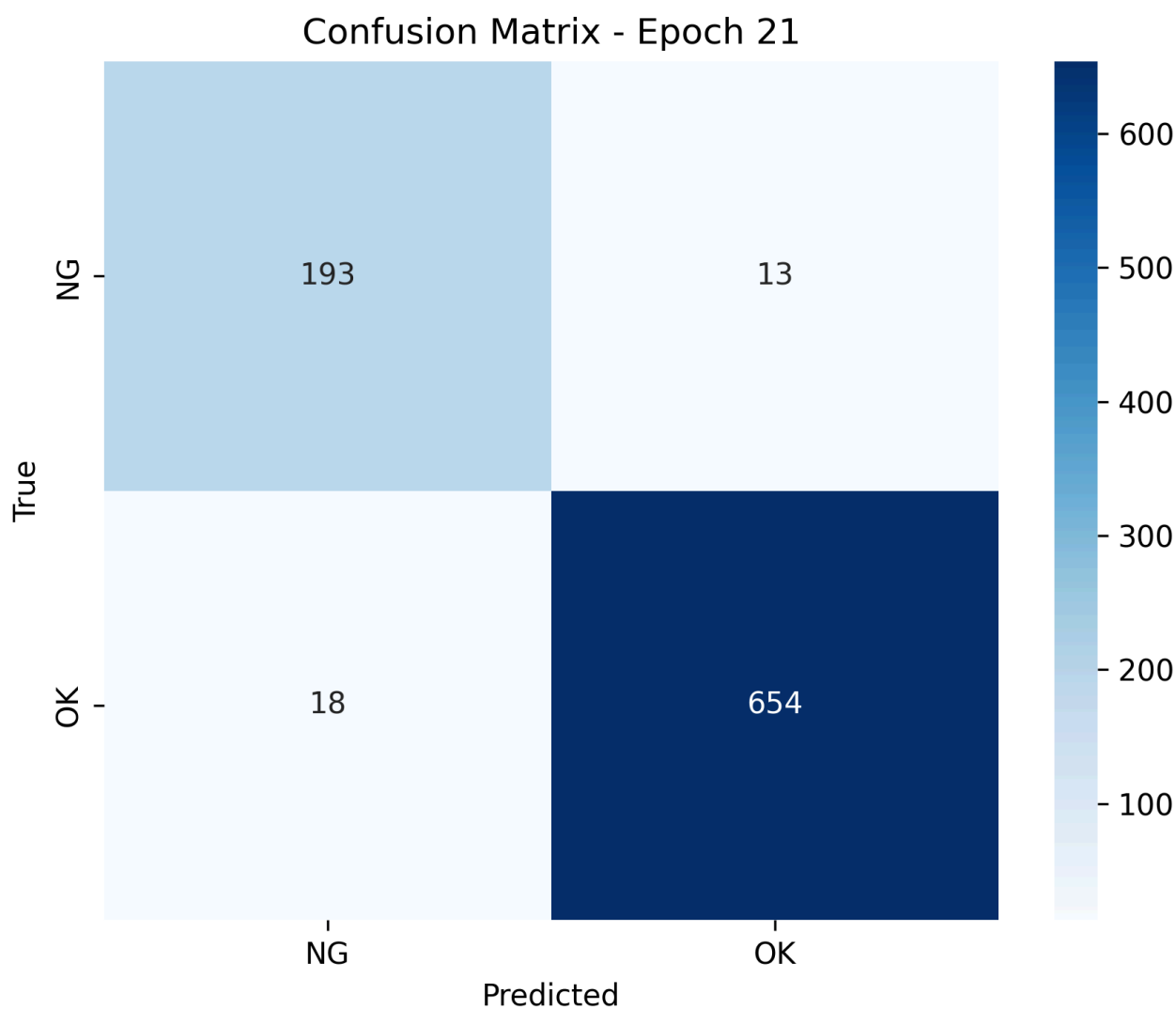
    torch.save(model.state_dict(), self.save_path)

early_stopper = EarlyStopping(patience=10, save_path='model_final.pth')
```

在第一次训练中，25轮后触发早停：



准确率在20轮左右出现峰值：



在测试集上进行测试：

	precision	recall	f1-score	support
0	0.8851	0.7549	0.8148	204
1	0.9425	0.9762	0.9591	840
accuracy			0.9330	1044
macro avg	0.9138	0.8655	0.8869	1044
weighted avg	0.9313	0.9330	0.9309	1044